

SmartDoc AI - Intelligent Document QA System

Nguyễn Nhật Hào*, Hoàng Anh Huy†, Phan Lập Thành‡

*Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn, †Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn,

‡Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn

Email: hao362k5@gmail.com, hoanganhhuy200499@gmail.com, phanthanh2775@gmail.com

GitHub: <https://github.com/nguyennhathao438/SmartDocAI.git>

Tóm tắt nội dung—Trong bối cảnh bùng nổ dữ liệu số, các Mô hình Ngôn ngữ Lớn (LLMs) đã chứng minh sức mạnh vượt trội nhưng vẫn vấp phải hai rào cản lớn: hiện tượng “ảo giác” thông tin và giới hạn tri thức. Bài báo cáo trình bày quá trình nghiên cứu và phát triển SmartDoc AI – một hệ thống hỏi đáp tài liệu thông minh ứng dụng ba kiến trúc tiên tiến: RAG, GraphRAG và CoRAG, hoạt động hoàn toàn cục bộ (local-first) trên mô hình Qwen2.5 và cơ sở dữ liệu FAISS nhằm đảm bảo quyền riêng tư dữ liệu. Dự án khai phá sự kết hợp giữa truy xuất vector, đồ thị tri thức và suy luận chuỗi tư duy để tạo ra giải pháp truy vấn chính xác, kiểm soát hallucination hiệu quả và tối ưu trải nghiệm hỏi đáp tài liệu tiếng Việt.

Keywords—RAG, GraphRAG, CoRag, LLM, FAISS.

I. CƠ SỞ LÝ THUYẾT

A. Retrieval-Augmented Generation (RAG)

1) **Khái niệm và kiến trúc**: Retrieval-Augmented Generation (RAG) là phương pháp kết hợp sức mạnh của Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) và hệ thống truy xuất thông tin, nhằm khắc phục hiện tượng “ảo giác” (hallucination) và giới hạn dữ liệu huấn luyện của LLM.

Kiến trúc cơ bản của RAG gồm hai thành phần chính:

- **Retrieval (Truy xuất)**: Tìm kiếm các tài liệu liên quan từ cơ sở dữ liệu bên ngoài.
- **Generation (Sinh văn bản)**: Sử dụng LLM để sinh câu trả lời dựa trên truy vấn đầu vào và ngữ cảnh được truy xuất.

Mô hình toán học của RAG được biểu diễn như sau:

$$RAG(x) = Generator(x, Retriever(x)) \quad (1)$$

Trong đó x là câu hỏi đầu vào của người dùng.

2) **Text Embedding và Vector Database**: Text Embedding là kỹ thuật mã hóa văn bản thành vector số học trong không gian đa chiều, cho phép máy

tính đo lường độ tương đồng ngữ nghĩa thay vì chỉ so sánh từ khóa.

$$E : \text{Text} \rightarrow \mathbb{R}^d \quad (2)$$

Các văn bản có ý nghĩa tương tự sẽ được ánh xạ thành các vector gần nhau trong không gian. Mức độ tương đồng thường được đo bằng Cosine Similarity.

Trong hệ thống, FAISS (Facebook AI Similarity Search) được sử dụng làm cơ sở dữ liệu vector, hỗ trợ tìm kiếm hiệu suất cao trên tập dữ liệu lớn, tối ưu bộ nhớ và có khả năng tăng tốc bằng GPU.

B. Graph Retrieval-Augmented Generation (GraphRAG)

1) **Khái niệm và kiến trúc**: GraphRAG là phương pháp mở rộng của RAG, khắc phục hạn chế trong việc tóm tắt toàn cục và kết nối các thông tin rời rạc bằng cách sử dụng Đồ thị tri thức (Knowledge Graph - KG).

Mô hình toán học mở rộng:

$$GraphRAG(x) = Generator(x, Retriever(x, G, C)) \quad (3)$$

Trong đó:

- G là đồ thị tri thức
- C là các báo cáo tóm tắt cộng đồng

Dữ liệu trong GraphRAG được biểu diễn dưới dạng:

$$G = (V, E, L) \quad (4)$$

Trong đó:

- V : tập các đỉnh (thực thể)
- E : tập các cạnh (mối quan hệ)
- L : tập nhãn hoặc thuộc tính

2) **Trích xuất và lập chỉ mục đồ thị:** Quá trình xây dựng đồ thị gồm ba bước chính:

- **Trích xuất:** Sử dụng LLM để nhận diện thực thể (Entity) và mối quan hệ (Relation) từ văn bản.
- **Chuẩn hóa:** Hợp nhất các thực thể trùng lặp hoặc đồng nghĩa nhằm tránh phân mảnh dữ liệu.
- **Lập chỉ mục:** Lưu trữ cấu trúc đồ thị và duy trì liên kết nguồn (Source Linking) để truy xuất ngược về tài liệu gốc.

C. Chain-of-Thought Retrieval-Augmented Generation (CoRAG)

1) **Khái niệm và kiến trúc:** CoRAG (Chain-of-Thought RAG) là mở rộng của RAG truyền thống, tích hợp cơ chế suy luận chuỗi tư duy (Chain-of-Thought – CoT) vào quá trình truy xuất và sinh câu trả lời. Thay vì truy xuất một lần, CoRAG phân rã câu hỏi thành nhiều bước suy luận trung gian, mỗi bước có thể kích hoạt một lần truy xuất độc lập.

Mô hình CoRAG được biểu diễn như sau:

$$\mathcal{R} = \{Retriever(x_i)\}_{i=1}^n \quad (5)$$

$$CoRAG(x) = Generator(x, \mathcal{R}, CoT) \quad (6)$$

Trong đó:

- x : câu hỏi gốc
- x_i : câu hỏi con tại bước i
- CoT : chuỗi suy luận trung gian
- n : số bước suy luận động

2) **Cơ chế suy luận đa bước:** CoRAG hoạt động theo vòng lặp suy luận – truy xuất – tổng hợp gồm ba bước:

- **Query Decomposition:** Phân rã câu hỏi thành $\{x_1, x_2, \dots, x_n\}$
- **Iterative Retrieval:** Truy xuất từng x_i trên FAISS để thu ngữ cảnh C_i
- **CoT Aggregation:** Tổng hợp $\{C_1, C_2, \dots, C_n\}$ và đưa vào prompt cuối để sinh câu trả lời

1) **Truy xuất đa bước (Graph Retrieval):** Khác với RAG truyền thống chỉ dựa trên độ tương đồng vector, GraphRAG thực hiện:

- **Entity Linking:** ánh xạ truy vấn vào các node trong đồ thị
- **Multi-hop Traversal:** duyệt qua nhiều bước trên các cạnh để thu thập thông tin liên quan

Cách tiếp cận này giúp mở rộng ngữ cảnh một cách có cấu trúc, cung cấp cho LLM một cái nhìn

tổng thể và liên kết, từ đó cải thiện độ chính xác của câu trả lời.

D. Mô hình ngôn ngữ lớn và nền tảng

1) **Mô hình Qwen2.5:** Hệ thống sử dụng mô hình Qwen2.5 của Alibaba Cloud, dựa trên kiến trúc Transformer hiện đại. Mô hình hỗ trợ cửa sổ ngữ cảnh dài (lên đến 128K tokens) và được tối ưu cho đa ngôn ngữ, bao gồm tiếng Việt.

Phiên bản 7B (7 tỷ tham số) được lựa chọn nhằm cân bằng giữa hiệu năng suy luận và khả năng triển khai trên phần cứng cục bộ.

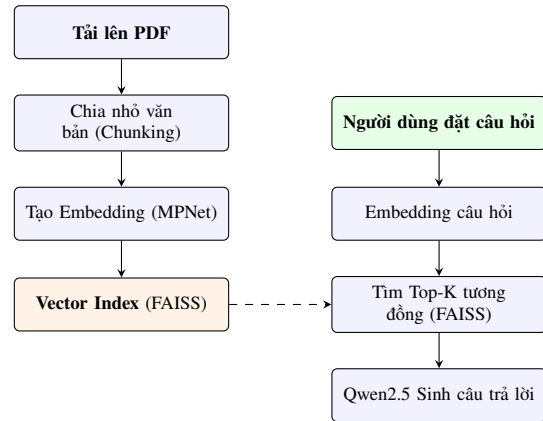
2) **Framework Ollama:** Ollama là nền tảng cho phép triển khai và vận hành LLM trên môi trường cục bộ (local-first), đảm bảo tính riêng tư và bảo mật dữ liệu. Hệ thống không yêu cầu kết nối internet trong quá trình xử lý.

E. Môi trường điều phối LangChain

LangChain là framework Python được sử dụng để kết nối các thành phần trong hệ thống như retriever, LLM và pipeline xử lý, giúp xây dựng luồng xử lý hoàn chỉnh cho ứng dụng.

II. THỰC NGHIỆM

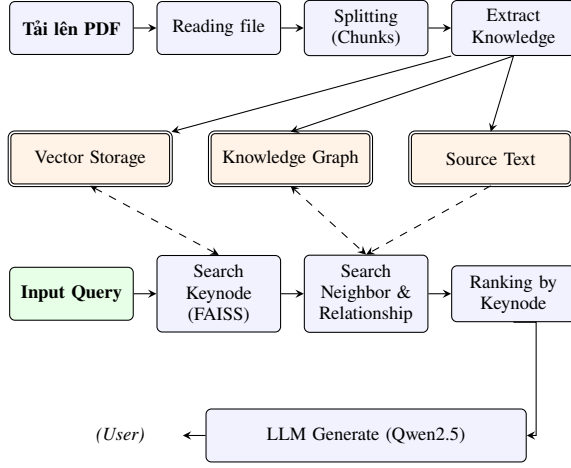
A. Luồng xử lý hệ thống RAG(Flowchart)



Hình 1. Luồng xử lý hệ thống RAG

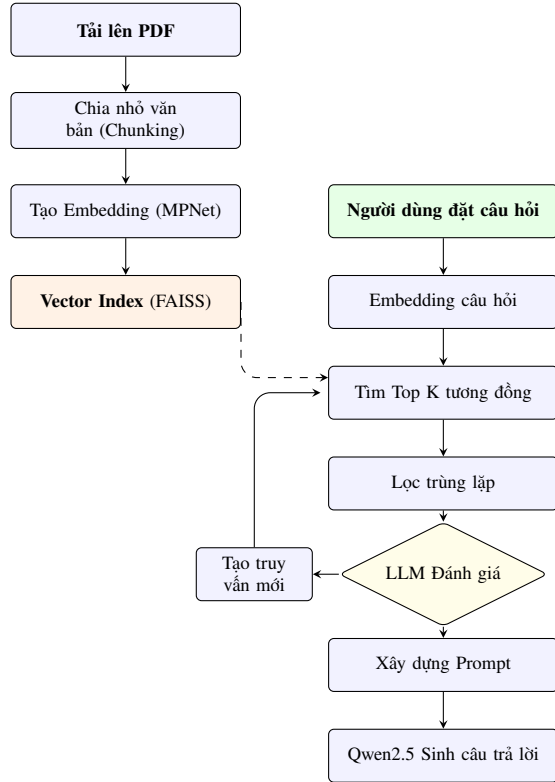
B. Luồng xử lý hệ thống GraphRAG (Flowchart)

Khác với RAG truyền thống, luồng GraphRAG bổ sung giai đoạn trích xuất tri thức dạng đồ thị để hiểu sâu hơn về mối liên kết giữa các thực thể trong tài liệu.



Hình 2. Luồng xử lý tri thức dựa trên đồ thị (GraphRAG) hoàn thiện

C. Luồng xử lý hệ thống coRAG(Flowchart)



Hình 3. Luồng xử lý Co-RAG SmartDoc AI

D. Phân tích độ phức tạp hệ thống (Theoretical Complexity)

Để đảm bảo tính nhất quán trong việc phân tích độ phức tạp thuật toán, các tham số hệ thống được

định nghĩa như sau:

- k : Số lượng ký tự của tài liệu đầu vào.
- c : Số lượng phân mảnh văn bản (chunks), với $c \propto k$.
- d : Số chiều của vector embedding (ví dụ: $d = 768$ cho mô hình *mpnet*).
- V, E : Lần lượt là tập hợp các thực thể (nodes) và quan hệ (edges) trong đồ thị tri thức $G = (V, E)$.
- n : Số lượng chunk được lựa chọn để trích xuất tri thức ($n \leq c$).
- e : Số lượng thực thể liên quan được truy xuất từ Vector Database trong quá trình truy vấn.
- V_{sub} : Số lượng cạnh trong đồ thị con được duyệt khi truy vấn.
- q : Số lượng cạnh trong đồ thị con được duyệt khi truy vấn.
- t : Số lượng truy vấn (query) trong một phiên suy luận hoặc quá trình đánh giá hệ thống.

1) Kiến trúc RAG truyền thống:

- **Giai đoạn Indexing:** $O_{idx}^{RAG} = O(k) + O(c \cdot d)$. Quá trình này chủ yếu tốn chi phí cho việc xử lý văn bản thô và chuyển đổi sang không gian vector.
- **Giai đoạn Retrieval:** $O_{ret}^{RAG} = O(d) + O(c \cdot d) + t$. Trong đó, $O(c \cdot d)$ là chi phí tìm kiếm tương đồng trên toàn bộ kho dữ liệu vector.

2) Kiến trúc GraphRAG:

- **Giai đoạn Indexing:** $O_{idx}^{GraphRAG} = O(k) + O(n \cdot t) + O(E + V)$. Đây là bước tốn kém nhất do phụ thuộc trực tiếp vào số lần gọi LLM ($n \cdot t$) để trích xuất tri thức.
- **Giai đoạn Retrieval:** $O_{ret}^{GraphRAG} = O(c \cdot d) + O((e + V_{sub}) \log(e + V_{sub})) + t$. Việc truy vấn tập trung vào các node liên quan và duyệt đồ thị con V_{sub} , giúp tối ưu hóa ngữ cảnh.

3) Kiến trúc CoRAG:

- **Giai đoạn Indexing:** $O_{idx}^{CoRAG} = O(k) + O(c \cdot d)$. Tương tự RAG, không yêu cầu xây dựng đồ thị hay tiền xử lý phức tạp.
- **Giai đoạn Retrieval:** $O_{ret}^{CoRAG} = q \cdot (O(d) + O(c \cdot d)) + t$, trong đó $q \leq 3$ là số lượng truy vấn (query) được thực hiện trong quá trình suy luận. Độ phức tạp tăng tuyến tính theo q , do mỗi truy vấn đều bao gồm truy xuất vector và gọi mô hình ngôn ngữ lớn.

E. Thực nghiệm trên tài liệu hướng dẫn và Đối chứng lý thuyết

Thực nghiệm được tiến hành trên tệp "Giới thiệu quy trình thiết lập card mạng LAN.pdf" để kiểm chứng các công thức độ phức tạp đã nêu.

1) **Thiết lập kịch bản kiểm thử:** Chúng tôi xây dựng tập dữ liệu thử nghiệm gồm một tài liệu có độ dài khoảng 2,300 ký tự, được chia thành 5 trang. Văn bản được phân mảnh bằng phương pháp chunking với kích thước mỗi đoạn là 500 ký tự và độ chồng lấn (overlap) là 50 ký tự nhằm đảm bảo tính liên kết ngữ nghĩa giữa các đoạn. Với thiết lập này, tổng số chunk thu được xấp xỉ $c \approx 5$.

Trên cơ sở đó, ba kịch bản kiểm thử được thiết lập như sau:

- **TC1 (Sự kiện đơn giản):** Truy vấn thông tin trực tiếp trong một chunk, đánh giá khả năng truy xuất cơ bản.
- **TC2 (Suy luận phức tạp):** Yêu cầu kết hợp nhiều chunk hoặc suy luận, đánh giá khả năng tổng hợp ngữ nghĩa.
- **TC3 (Ngoài phạm vi):** Câu hỏi không liên quan tài liệu, đánh giá khả năng kiểm soát nhiễu và hạn chế hallucination.

Bảng I
KẾT QUẢ THỰC NGHIỆM ĐỐI CHỨNG THỜI GIAN XỬ LÝ (RETRIEVAL)

Kịch bản	Kiến trúc	Thời gian (s)	Nhận xét kết quả
TC1	RAG	45s	Đúng, truy xuất trực tiếp.
	GraphRAG	225s	Bao quát hơn RAG.
	CoRAG	93s	Đúng, giải thích từng bước rõ ràng.
TC2	RAG	45s	Giải thích được một phần.
	GraphRAG	225s	Tương đối đủ nhưng chưa thuyết phục.
	CoRAG	190s	Đầy đủ hơn, suy luận có cấu trúc logic.
TC3	RAG	45s	Đúng với mong đợi.
	GraphRAG	225s	Đúng nhưng đôi khi vẫn bịa đặt.
	CoRAG	190s	Từ chối trả lời rõ ràng, ít hallucination nhất.

- 1) **Sự đánh đổi về Indexing:** Đúng như lý thuyết $n \cdot t$, GraphRAG tốn chi phí tiền xử lý rất cao khi dùng mô hình local ($t \approx 45s$), tuy nhiên chi phí này chỉ trả một lần. CoRAG không phát sinh chi phí Indexing bổ sung so với RAG, do không yêu cầu xây dựng đồ thị tri thức.
- 2) **Hiệu quả truy vấn (Retrieval):**

- Thời gian truy vấn của RAG ổn định ở mức 45s, GraphRAG 225s. CoRAG dao động trong khoảng 93s–190s tùy theo số bước suy luận n được sinh ra động, định vị ở mức trung gian giữa hai kiến trúc.
- Về chất lượng, RAG dễ bị nhiễu do truy xuất toàn bộ chunk ($O(c \cdot d)$). GraphRAG cải thiện tính liên kết nhờ duyệt đồ thị con (V_{sub}). CoRAG nổi bật ở khả năng kiểm soát hallucination tốt nhất (TC3) và suy luận có cấu trúc với câu hỏi phức tạp (TC2) thông qua cơ chế truy xuất lặp $n \cdot O(c \cdot d)$.

- 3) **Kết luận về sự đánh đổi (Trade-off):** RAG ưu tiên tốc độ và triển khai đơn giản. GraphRAG đánh đổi chi phí Indexing cao để lọc nhiễu tốt hơn. CoRAG giữ Indexing nhẹ như RAG nhưng tăng chi phí Retrieval theo n , phù hợp với các truy vấn đòi hỏi suy luận sâu mà GraphRAG chưa xử lý mạch lạc về mặt logic.

F. Thực nghiệm tối ưu mô hình

Thực nghiệm thứ hai được tiến hành trên tệp "rag_doc.pdf", là một tài liệu nghiên cứu chuyên sâu về kiến trúc Retrieval-Augmented Generation (RAG). Đây là một tập dữ liệu có cấu trúc phức tạp, chứa nhiều lớp thông tin liên kết, đòi hỏi hệ thống phải có khả năng bao quát ngữ nghĩa tốt và duy trì tính nhất quán trong truy xuất tri thức.

Mục tiêu của thực nghiệm là đánh giá ảnh hưởng của các tham số chunk size và overlap đến hiệu năng và thời gian phản hồi của ba kiến trúc: RAG truyền thống, GraphRAG và CoRAG.

Các cấu hình chunk/overlap được sử dụng bao gồm:

- 400/40
- 800/80
- 800/160
- 1200/120

Kết quả đối với RAG cho thấy thời gian xử lý gần như không thay đổi đáng kể giữa các cấu hình, duy trì trung bình khoảng ~ 7 giây cho mỗi truy vấn. Tuy nhiên, khi kích thước chunk tăng, chất lượng câu trả lời có xu hướng cải thiện do khả năng bao phủ ngữ cảnh tốt hơn, giúp giảm hiện tượng thiếu thông tin trong truy xuất.

GraphRAG thể hiện sự biến thiên rõ rệt về thời gian xử lý theo số lượng chunk. Cụ thể:

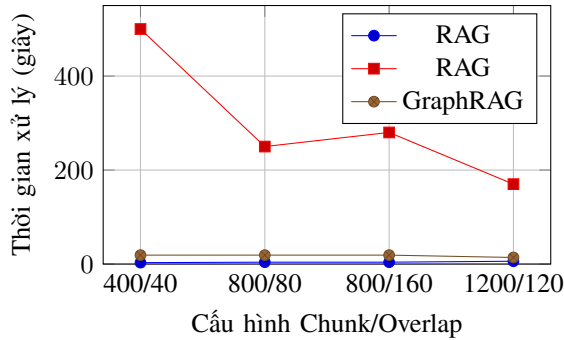
- 400/40: $\sim 500s$
- 800/80: $\sim 250s$

- 800/160: ~280s
- 1200/120: ~170s

CoRAG cũng cho thấy xu hướng biến thiên tương tự GraphRAG nhưng ở mức độ thấp hơn đáng kể, do không phát sinh chi phí xây dựng đồ thị tri thức. Cụ thể:

- 400/40: ~19s
- 800/80: ~19s
- 800/160: ~19s
- 1200/120: ~14s

Sự giảm thời gian khi tăng chunk size ở cả GraphRAG và CoRAG có thể được giải thích bởi việc giảm số lượng đơn vị cần xử lý, từ đó giảm số bước suy luận n và lần gọi mô hình tương ứng.



Hình 4. So sánh thời gian xử lý giữa RAG, GraphRAG và CoRAG theo cấu hình chunk/overlap

Tổng hợp kết quả cho thấy ba kiến trúc phân tầng rõ ràng về chi phí tính toán. RAG duy trì thời gian phản hồi thấp nhất (3–6s) và ổn định nhất qua các cấu hình. CoRAG định vị ở mức trung gian (14–19s), gần với RAG hơn GraphRAG, nhờ bỏ qua bước xây dựng đồ thị trong khi vẫn duy trì khả năng suy luận chuỗi. GraphRAG có chi phí cao nhất (170–500s) do phụ thuộc trực tiếp vào số lần gọi LLM $n \cdot t$ trong giai đoạn trích xuất thực thể.

III. ĐÁNH GIÁ TỔNG QUÁT

- 1) **Sự khác biệt kiến trúc:** RAG dựa trên truy xuất vector toàn cục, có ưu điểm về tốc độ ổn định và chi phí thấp. GraphRAG khai thác cấu trúc đồ thị tri thức để thu hẹp không gian tìm kiếm, cải thiện liên kết ngữ nghĩa. CoRAG bổ sung cơ chế phân rã câu hỏi và truy xuất lặp, cho phép suy luận có cấu trúc mà không yêu cầu xây dựng đồ thị.
- 2) **Ảnh hưởng của tiền xử lý:** Hiệu năng RAG ít phụ thuộc vào cấu hình phân mảnh văn bản. GraphRAG phụ thuộc mạnh vào số lượng

chunk do chi phí trích xuất thực thể và xây dựng đồ thị. CoRAG có mức phụ thuộc trung bình – chunk size lớn hơn giúp giảm số bước suy luận n , từ đó rút ngắn thời gian phản hồi.

- 3) **Chất lượng kết quả:** RAG dễ bị nhiễu khi truy xuất toàn bộ không gian vector. GraphRAG cải thiện tổng hợp thông tin nhờ quan hệ thực thể nhưng đôi khi thiếu mạch lạc logic. CoRAG nổi bật ở khả năng suy luận chuỗi nhân quả và kiểm soát hallucination tốt nhất trong ba kiến trúc.
- 4) **Kiểm soát đầu ra:** GraphRAG dùng cấu trúc đồ thị như cơ chế ràng buộc để giảm thông tin sai lệch, hiệu quả phụ thuộc chất lượng tiền xử lý. CoRAG kiểm soát đầu ra thông qua từng bước suy luận trung gian, giúp mô hình tự giới hạn phạm vi trả lời rõ ràng hơn so với hai kiến trúc còn lại.
- 5) **Sự đánh đổi tổng thể:** RAG phù hợp với hệ thống yêu cầu tốc độ và triển khai đơn giản. GraphRAG phù hợp với bài toán cần khai phá quan hệ thực thể, chấp nhận chi phí Indexing cao. CoRAG là lựa chọn cân bằng cho các truy vấn phức tạp đòi hỏi suy luận sâu, với chi phí Indexing thấp nhưng Retrieval tăng tuyến tính theo số bước n .

IV. KẾT LUẬN

Thông qua quá trình nghiên cứu và thực hiện đồ án SmartDoc AI, chúng em đã đạt được những mục tiêu quan trọng sau:

- **Về kiến thức:** Nắm vững cơ chế hoạt động của ba kiến trúc RAG, GraphRAG và CoRAG; hiểu cách chuyển đổi dữ liệu phi cấu trúc (PDF) thành không gian vector tri thức thông qua mô hình Embedding (MPNet) và cơ chế suy luận chuỗi tư duy.
- **Về kỹ thuật:** Triển khai thành công hệ thống trên môi trường cục bộ sử dụng mô hình Qwen2.5:7b, đảm bảo tính bảo mật và hỗ trợ tốt cho tiếng Việt. Làm chủ việc quản lý Vector Database với FAISS và tích hợp CoRAG với cơ chế truy xuất lặp đa bước.
- **Điểm cần cải thiện:** Thời gian xử lý với tài liệu lớn (trên 100 trang) vẫn còn độ trễ, đặc biệt ở CoRAG khi số bước suy luận n tăng cao. Hiện tượng hallucination vẫn xuất hiện ở GraphRAG với câu hỏi ngoài phạm vi tài liệu, dù CoRAG đã cải thiện đáng kể khả năng từ chối trả lời trong các trường hợp này.

V. HƯỚNG PHÁT TRIỂN (CÔNG VIỆC THẮNG TỐI)

Dựa trên kết quả thực nghiệm và các yêu cầu mở rộng hệ thống, kế hoạch trong giai đoạn tiếp theo tập trung vào việc tối ưu hóa hiệu năng và triển khai các kỹ thuật Advanced RAG:

- **Chủ đề nghiên cứu:** Tối ưu hóa truy xuất đa tầng (Hybrid Search), nén mô hình (Quantization) và cơ chế tự đánh giá (Self-RAG).
- **Mục tiêu cụ thể:**
 - 1) **Nâng cấp Hybrid Search:** Kết hợp tìm kiếm ngữ nghĩa (Vector Search) với tìm kiếm từ khóa (BM25) thông qua cơ chế *Ensemble Retriever* nhằm cải thiện độ chính xác khi truy vấn các thuật ngữ chuyên biệt.
 - 2) **Tích hợp Re-ranking với Cross-Encoder:** Bổ sung bước đánh giá lại mức độ liên quan của tài liệu sau khi truy xuất bằng mô hình Cross-Encoder để lọc nhiễu, đồng thời tối ưu hóa độ trễ (latency) cho toàn quy trình.
 - 3) **Triển khai Conversational RAG:** Xây dựng bộ nhớ (Memory) để theo dõi ngữ cảnh cuộc hội thoại, giúp LLM có khả năng tham chiếu và xử lý tốt các câu hỏi nối tiếp (follow-up questions).
 - 4) **Quản lý đa tài liệu và Metadata:** Hỗ trợ tải lên nhiều PDF cùng lúc, thực hiện lọc theo thuộc tính (Metadata filtering) như tên file, ngày tải, và hiển thị rõ ràng nguồn gốc thông tin từ tài liệu nào.
 - 5) **Cơ chế Citation và Source Tracking:** Hiển thị chính xác vị trí nguồn gốc thông tin (trang số, ngữ cảnh), cho phép người dùng click để xem đoạn văn gốc đã được highlight trong PDF.
 - 6) **Nghiên cứu Self-RAG và Quantization:** Triển khai cơ chế để LLM tự đánh giá câu trả lời (Self-reflection) và tự động cải thiện câu hỏi (Query rewriting). Đồng thời áp dụng Quantization (4-bit/8-bit) để giảm tải tài nguyên RAM khi chạy thực tế.

VI. TÀI LIỆU THAM KHẢO

TÀI LIỆU

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] D. Edge et al., “From Local to Global: A GraphRAG Approach to Query-Focused Summarization,” *Microsoft Research*, Tech. Rep., 2024. [Online]. Available: arXiv:2404.16130.
- [3] Qwen Team, “Qwen2.5: A Party of Foundation Models,” *Alibaba Cloud*, 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [4] LangChain Inc., “LangChain: Framework for developing applications powered by language models,” 2023. [Online]. Available: <https://python.langchain.com/>
- [5] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [6] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: guaranteeing well-connected communities,” *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.